

L&T TECH XCELERATE

Team Name : fortrans

Domain 1 : Autonomous Navigation & Battlefield Mobility

Robust Zone Mapping & Navigation Rover, Build a basic rover capable of autonomous waypoint navigation in diverse terrains with highly constrained compute hardware.

Literature Survey

The military robotics and autonomous systems market has grown significantly, with current valuations of \$19.68 billion in 2024 and projections reaching \$32.50 billion by 2030, reflecting a CAGR of 8.7% (source). This expansion is fueled by increased demand for autonomous surveillance, reconnaissance, logistics support, and unmanned operations in complex environments.

Simultaneous Localization and Mapping (SLAM) for Outdoors

Simultaneous Localization and Mapping (SLAM) is fundamental for autonomous systems, enabling a robot to construct a map of unknown terrain while tracking its own position. Outdoor SLAM is more complex than indoor operations due to vast, dynamic environments with moving obstacles, variable lighting, and changing weather. A 360° LIDAR sensor provides reliable solutions for outdoor SLAM. The sensor continuously emits laser pulses in all directions, generating a point cloud that accurately represents the surrounding environment, independent of lighting conditions. Point cloud scans are compared over time, employing scan matching to estimate robot motion. To mitigate cumulative errors, or drift, loop closure methods correct maps when revisiting known locations. The primary advantages of a 360° LIDAR include complete coverage and long-range detection, facilitating effective path planning even in open areas exceeding 100 meters.

Greedy Best First Search (GBFS)

GBFS is a heuristic-based pathfinding algorithm that prioritizes the node closest to a goal at each step, typically using Euclidean distance. It begins with the start node in a priority queue, expands neighbors based on estimated distance, and iterates until the target is reached. GBFS offers speed by directing navigation toward the goal, although it does not guarantee the shortest path due to ignoring actual traversal cost. In robotic navigation, GBFS is ideal for open outdoor environments where rapid pathfinding outweighs absolute optimality, with A* or D* Lite preferred in complex terrains.

Sensor Usage in Autonomous Rovers

Autonomous rovers in outdoor or battlefield environments require multi-sensor systems for robust perception. A combination of IMU, TOF distance sensors, GPS, and FPV cameras ensures continuous operation under uncertain conditions.

1. **IMU (Inertial Measurement Unit):** Tracks roll, pitch, and yaw using a 3-axis accelerometer, gyroscope, and sometimes a magnetometer. Sensor fusion via complementary or Kalman filters enables accurate dead reckoning and orientation even when GPS signals degrade.
2. **TOF Sensor:** Detects potholes and terrain irregularities by measuring infrared light return time, enhancing safety in uneven environments.
3. **GPS Module:** Provides waypoint navigation, converting latitude and longitude into navigable coordinates. When fused with IMU data, the rover maintains accurate heading even with basic GPS accuracy.
4. **FPV Camera:** Streams real-time video for operator supervision and visual verification of terrain and obstacles. FPV feeds can also support computer vision algorithms for obstacle detection and landmark identification.

Combining these sensors creates redundancy, improves reliability, and allows the rover to navigate dynamically in unmapped regions.

Unique Selling Proposition

The rover is designed for compactness, autonomy, and cost-effectiveness. At ₹25,000 and within a 30 cm cube, it balances portability and capability. Its sensor fusion stack ensures robust navigation, pothole detection, and live visual feedback. Compact size allows rapid monitoring, stealthy deployment, and quick observation of tight environments inaccessible to larger systems. Modular design facilitates maintenance, future upgrades, and potential swarm deployment.

System Architecture

Overview

The rover employs a four-wheel differential drive system, with left and right motor pairs controlled by separate Smartelex 15S DC motor drivers. The Raspberry Pi 4B 8GB serves as the central compute unit, handling navigation, SLAM, sensor fusion, and high-level decision-making. An Arduino microcontroller manages real-time motor control, encoder feedback, and safety features. The sensor suite includes the YDLIDAR X2 360° scanner, VL53L0X TOF sensor, FPV camera, IMU, and GPS.

Physical Placement and Mechanical Notes

- **Chassis:** Sized to 25x12 cm for compact deployment, maintaining stability and allowing wheel overhang.
- **LIDAR:** Mounted atop the chassis for unobstructed 360° scanning.
- **FPV Camera:** Front-mounted with adjustable tilt.
- **TOF Sensor:** Low front placement angled downward for pothole detection.
- **IMU:** Near the center of mass to minimize vibration effects.
- **Motors:** Two 300 rpm gear motors per side, paired with 130 mm wheels.
- **Motor Drivers:** Mounted centrally for efficient wiring and cooling.
- **Raspberry Pi:** Metal case with dual fans to ensure thermal stability.

This layout balances weight distribution, accessibility, and modularity, enhancing stability and ease of maintenance.

Electrical Architecture and Power Distribution

A dual Li-ion battery pack powers motors and electronics. Power distribution includes fused branches for each driver and a 5V rail via DC-DC buck converters for sensitive electronics. Star grounding minimizes electrical noise. Voltage monitoring and current sensing protect the system and trigger failsafe operations if required.

Motor Control Hardware and Wiring

Each motor pair is wired in parallel to a Smartelex driver. Fuses protect against overcurrent. Encoders provide velocity feedback to the Arduino, which implements a PID loop for precise control. Acceleration ramping reduces mechanical shock and current spikes.

Low-Level Control

The Arduino handles PWM generation, direction signals, encoder reading, and emergency stops. It communicates with the Raspberry Pi through a serial protocol, receiving target velocities and reporting measured values.

High-Level Compute on Raspberry Pi 4B

The Raspberry Pi runs ROS 2 Humble, with nodes responsible for:

- Navigation (GBFS algorithm)
- SLAM (Hector SLAM or GMapping)
- Sensor fusion (IMU, GPS, odometry)
- FPV processing (OpenCV)
- Obstacle detection (LIDAR and TOF)

- Motor control interface to Arduino

CPU usage is kept below 60% for thermal headroom. The dual-fan metal case maintains optimal performance during intensive computation.

Perception and Mapping

LIDAR scans are preprocessed for motion correction using IMU data. A local occupancy grid is maintained for path planning. TOF sensors detect sudden terrain drops, raising virtual obstacles in the map to prevent collisions. FPV cameras provide visual verification of terrain and obstacles.

Safety Features

- Hardware kill switch
- Battery low triggers safe return
- Overcurrent detection
- Arduino watchdog timer
- Black box logging for post-mission analysis

Thermal and Electromagnetic Considerations

Motor power lines are routed separately from low-voltage sensor lines. Smartex drivers have airflow clearance, and ferrite beads mitigate EMI on USB camera cables.

Software Reliability and Testing Plan

Unit tests for sensors and motor interfaces, hardware-in-loop simulations, and progressive field tests ensure reliable deployment. Metrics include waypoint accuracy, pothole detection, battery life, and peak current draw.

Packaging and Deployability

Modular internal layout allows quick replacement of motors, drivers, and sensors. Shock mounts and protective casing enhance durability, while compact dimensions facilitate covert transport.

Methodology

Mechanical Design and Component Layout

The chassis supports a compact footprint, with motor pairs on each side in a tank-style arrangement. The center of mass is optimized to balance forward sensors and rear batteries. LIDAR, FPV camera, TOF sensors, IMU, and GPS are strategically positioned to minimize wiring complexity and maximize sensor coverage.

Power Management

Dual Li-ion batteries supply motors and electronics. Fused branches and buck converters regulate voltage, with monitoring circuits for safety. Star grounding reduces noise, and the architecture supports emergency stops and fault handling.

Software Architecture

The rover employs sensor fusion, SLAM-based navigation, and GBFS waypoint planning. LIDAR detects obstacles, TOF sensors detect terrain hazards, and the FPV camera provides visual confirmation. Differential drive PID loops ensure accurate trajectory control.

Exploration and Unmapped Navigation

Combining GPS-guided waypoints with real-time obstacle detection allows autonomous exploration of unmapped regions. Compact design ensures rapid deployment, modular maintenance, and portability. Covert transport is achievable due to the small form factor.

Execution and Engagement Plan

1. **Phase 1 – Software Development and Component Finalization:** Software algorithms are finalized while mechanical and electronic layouts are optimized.
2. **Phase 2 – Hardware-Software Integration:** Components are connected, verified, and preliminary tests are conducted in controlled conditions.
3. **Phase 3 – Field Testing and Demonstration:** Real-world evaluation of waypoint navigation, obstacle detection, and FPV streaming ensures readiness for the hackathon.

Timely receipt of hardware components is essential to meet deadlines.

Market Analysis, Business Linkage, and Measurable Impact

Market Analysis

The global AMR market was valued at USD 2.8 billion in 2024 and is projected to grow at a CAGR of 17.6% through 2034. In India, the market was USD 257.5 million in 2024, expected to reach USD 795.6 million by 2030. Growth is driven by e-commerce, warehouse automation, agriculture, and defense applications.

Business Linkage

- **E-commerce and Logistics:** AMRs automate material handling, reducing labor costs.
- **Manufacturing:** Facilitate material transport between stations.
- **Healthcare:** Transport medications and supplies.
- **Agriculture:** Assist with planting, harvesting, and monitoring.
- **Defense and Security:** Conduct surveillance, reconnaissance, and supply support.

Measurable Impact Indicators

- Operational efficiency improvements
- Cost savings via labor reduction
- Productivity gains from task automation
- Enhanced workplace safety
- Scalable deployment for various applications

Gap Analysis

- **Size and Portability:** Most platforms are bulky; this rover fits a 30 cm cube.
- **Cost Inefficiency:** Existing solutions are expensive; this design is ₹

25,000.

- **All-Terrain Performance:** Tank-style differential drive with high-traction wheels improves mobility.
- **Sensor Integration:** Multi-sensor fusion with LIDAR, TOF, GPS, IMU, and FPV camera ensures robust perception.
- **Navigation and Autonomy:** Combines GBFS, LIDAR detour planning, and terrain classification for adaptive exploration.

Technology Stack

1. **Computing Platform:** Raspberry Pi 4B 8GB running Ubuntu 22.04 LTS.
2. **Middleware:** ROS 2 Humble Hawksbill.
3. **Sensor Integration:** YDLIDAR X2L, VL53L0X, IMU, GPS, FPV camera.
4. **Mobility:** Tank-style differential drive with 300 rpm motors and 130 mm wheels.

5. **Power Management:** Dual LiPo batteries with buck converters.
6. **Algorithms:** GBFS, SLAM, sensor fusion, obstacle classification.
7. **Software Tools:** Python, C++, OpenCV, NumPy, SciPy, Rviz2, Gazebo.
8. **Advantages:** Modular, low-cost, compact, capable of autonomous navigation in unmapped outdoor terrains, and ready for future AI or swarm integration.

Hardware Justification

- **Raspberry Pi 4B:** Adequate computational power for multi-sensor processing.
- **360° LIDAR:** Provides real-time mapping and dynamic obstacle detection.
- **TOF Sensor:** Detects terrain irregularities for safe traversal.
- **IMU & GPS:** Ensures accurate localization and navigation.
- **FPV Camera:** Enables remote monitoring and visual confirmation.
- **Motor Drivers:** Two Smartelex 15S DC units provide reliable torque and modularity.
- **Motors and Wheels:** Tank-style drive ensures all-terrain mobility.
- **Power System:** Dual LiPo batteries and buck modules provide regulated energy.
- **Chassis:** Compact, rigid, and weight-optimized for stability.
- **Integration:** Modular placement facilitates sensor fusion and autonomous exploration.

Here's a detailed report with a dedicated paragraph for each component you listed. I've structured it to highlight the component, its specifications, and its usage in a robotics/autonomous system context.

Report on Usage of Robotics Components

Raspberry Pi 4 Model-B with 8 GB RAM

The **Raspberry Pi 4 Model-B** with 8 GB RAM is the central computing unit for most advanced robotics and autonomous systems. It offers a quad-core ARM Cortex-A72 CPU capable of running at 1.5 GHz and supports high-speed connectivity including USB 3.0 and Gigabit Ethernet. The 8 GB RAM allows for intensive computation tasks such as running ROS (Robot Operating System), LiDAR data processing, image recognition algorithms, and path planning simultaneously. In autonomous robots, it functions as the main controller, integrating sensor inputs, executing AI algorithms, and sending control signals to actuators and motors.

Raspberry Pi Metal Aluminium Case with Double Fans for Raspberry Pi 4B (Black)

The **metal aluminium case** with dual fans provides thermal management and physical protection to the Raspberry Pi 4B. High-performance computing, especially when running multiple processes such as computer vision and LiDAR scanning, generates substantial heat. The aluminium construction dissipates heat efficiently, while the dual fans actively cool the processor and RAM, preventing thermal throttling. This ensures stable performance for long-duration robotic operations and enhances the longevity of the Raspberry Pi under continuous load.

Sandisk Micro SD Card 64GB A1 Class

The **official 64GB A2 Class micro SD card** serves as the primary storage for the Raspberry Pi's operating system and applications. The A2 classification ensures fast random read/write performance, critical for loading large datasets, running multiple processes, and storing sensor data. In robotics projects, it can store ROS workspaces, pre-trained AI models, sensor logs, and firmware updates. Its unprogrammed state allows developers to flash a custom OS like Raspberry Pi OS or Ubuntu for robotics, ensuring flexibility in software deployment.

YDLIDAR X2 2D LiDAR Sensor – 360° Scanning, 8m Distance, ROS Compatible

The **YDLIDAR X2 2D LiDAR sensor** provides 360-degree scanning capability for autonomous navigation. With an 8-meter maximum range and ROS compatibility, it enables the robot to map its environment, detect obstacles, and generate real-time spatial data. LiDAR data is crucial for simultaneous localization and mapping (SLAM), allowing the robot to plan paths and navigate dynamically changing environments. Its high scanning frequency and ROS integration make it suitable for indoor and moderately complex outdoor robotic applications.

VL53L0X TOF Based LIDAR Laser Distance Sensor

The **VL53L0X Time-of-Flight (TOF) distance sensor** is a compact and precise sensor for measuring short-range distances up to 2 meters. It emits a laser pulse and calculates distance based on the time taken for the pulse to return. This sensor is ideal for obstacle detection in tight spaces, height measurement, and proximity-based triggering of robotic actions. Due to its small size, it can be mounted at multiple points on the robot to provide local sensing, complementing longer-range LiDAR sensors for complete situational awareness.

1/4 CMOS 640x480 USB Camera with Collapsible Cable for Raspberry Pi

The **1/4 CMOS USB camera** provides video input for computer vision applications. Its 640x480 resolution is sufficient for object recognition, line following, and simple navigation tasks. The collapsible cable ensures flexible installation on mobile platforms, while USB connectivity allows easy integration with the Raspberry Pi. In robotics, this camera can capture real-time images for processing with OpenCV or deep learning models, enabling tasks such as obstacle recognition, visual SLAM, and gesture-based control.

uBlox Neo M8N GPS Module M5T GLONASS with Antenna

The **uBlox Neo M8N GPS module** provides accurate geolocation data using both GPS and GLONASS satellites. With an external antenna, it enhances signal reception even in urban environments with limited satellite visibility. In autonomous robots, this module enables outdoor navigation, waypoint tracking, and geofencing. GPS data can also be fused with IMU and LiDAR inputs to improve positioning accuracy, especially in open-area operations where visual or LiDAR-only navigation may be insufficient.

Johnson 12V 300 RPM Offside Metal Gearbox Motor

The **GM37RH gearbox motor** is a high-torque, low-speed motor ideal for driving robotic wheels. Operating at 12V with 300 RPM, it provides precise rotational control for smooth movement. The metal gearbox ensures durability and consistent torque under load, which is critical for mobile robots carrying heavy sensors or batteries. These motors are typically connected to the Raspberry Pi via motor drivers, allowing software-controlled speed modulation, direction control, and integration with navigation algorithms.

130mm Robot Wheel for ATV and DIY Robotics

The **130mm robot wheels** provide the physical interface between motors and the robot's movement. Their size and material offer stability and traction on varied surfaces. In combination with GM37RH motors, they enable efficient conversion of rotational torque into linear movement. Proper wheel selection affects the robot's speed, maneuverability,

and payload capacity. For differential drive robots, accurate wheel diameter and mounting are essential for precise odometry calculations and reliable path planning.

11.1V 2200mAh LiPo Battery With IMax B3 LiPo Balance Charger

The **11.1V 2200mAh LiPo battery** serves as the main power source for mobile robots. LiPo batteries provide high energy density, allowing compact storage with sufficient voltage to drive motors, sensors, and computing units. The included IMax B3 charger ensures safe, balanced charging, maintaining the battery's health over repeated cycles. In robotic applications, battery selection impacts runtime, operational reliability, and safety, making high-quality LiPo batteries essential for extended autonomous missions.

XT60 Connector Male Female Pair

The **XT60 connectors** provide a secure, high-current connection between batteries and electronics. Rated for up to 60A, they are commonly used in LiPo battery systems to prevent accidental disconnections and minimize resistance in power delivery. In robotic projects, these connectors ensure stable power to motors, sensors, and controllers, reducing voltage drops and protecting components from sparks or short circuits during assembly and maintenance.

M3 30mm SS Screw (25 Pcs)

The **M3 30mm stainless steel screws** are essential for mechanical assembly of robot frames, motor mounts, and sensor brackets. Stainless steel offers corrosion resistance and mechanical strength, ensuring long-term durability in indoor and outdoor environments. Proper screw selection ensures tight assembly without stripping threads, contributing to overall structural stability of the robot platform.

EasyMech SS 304 Pan Philips Head M3 x 5 mm Bolt – 25 Pcs

The **EasyMech SS 304 pan Philips head bolts** are used to attach smaller components such as sensors, cameras, or electronic modules. Their pan head design provides a larger bearing surface, preventing damage to plastic or thin metal mounting plates. Stainless steel 304 ensures corrosion resistance and reliability, making these bolts a staple in precise robotic assembly, especially for mounting delicate or vibration-sensitive components.

EasyMech M3 SS Hex Nut – 50 Pcs

M3 stainless steel hex nuts complement screws and bolts in mechanical assemblies, securing components with threaded connections. Hex nuts allow precise torque application, preventing loosening during robot operation. They are commonly paired with washers and screws to form reliable, vibration-resistant fasteners. In robotics, they are used across chassis assembly, wheel mounting, and sensor installation.

Smartelex 15s Single Channel – Quantity:2

It handles 15A easily without requiring heatsinks.

Rpi Shop - M3 Flat Washer, Zinc Plated Quantity - 25 Pcs

M3 zinc-plated flat washers distribute the load of screws and bolts, reducing localized stress on mounting surfaces. They prevent deformation of thin materials such as acrylic, plastic, or aluminium plates used in robot frames. Washers also help in ensuring tight assembly without excessive torque, contributing to the mechanical stability and longevity of the robot's structure.

Freemans LONG NOSE PLIER

Long nose pliers are a versatile hand tool used during robotic assembly and wiring. They allow precision gripping, bending, and cutting of wires and small components in tight spaces. In robotics projects, long nose pliers are particularly useful for routing sensor cables, securing connectors, and making minor mechanical adjustments, making them an essential tool in both prototyping and maintenance.

Johnson/Mini Johnson Gear Motor Mount L Clamp (Bracket) – 4 pcs

The **Johnson motor mount L clamps** provide a stable platform to secure motors to the robot frame. These brackets ensure precise alignment of motor shafts with wheels or gears, minimizing mechanical stress and vibrations. Reliable motor mounting is critical in autonomous robots to maintain consistent wheel rotation, accurate odometry, and smooth navigation, especially when using geared motors like the GM37RH.

24V/12V to 5V 5A Power Module DC-DC XY-3606 Power Converter

The **XY-3606 DC-DC power module** converts higher voltage input from LiPo or battery packs to stable 5V output required by electronics such as Raspberry Pi, sensors, and communication modules. With a 5A output capacity, it can handle multiple devices simultaneously without voltage drops. Power converters like this are crucial in robotics to ensure all components receive reliable voltage regardless of battery fluctuations during movement. It is specially used for powering Raspberry Pi 4B.

LM2596S DC-DC Buck Converter Power Supply

The **LM2596S buck converter** provides adjustable voltage regulation from higher DC inputs to lower voltages, protecting sensitive electronics. It is often used to supply specific voltages to sensors, microcontrollers, or peripheral devices on the robot. Its compact size and efficiency make it ideal for embedded robotics, reducing heat generation while maintaining stable operation across various load conditions.

Comprehensive Work Update Report: ROS2 Humble Configuration for YDLiDAR Systems on Raspberry Pi OS

The documented process represents an extensive technical implementation of Robot Operating System 2 Humble distribution specifically configured for dual YDLiDAR sensor systems on a Raspberry Pi 4B 8GB running Debian Trixie-based Raspberry Pi OS. This comprehensive implementation involved establishing a complete development environment through systematic installation procedures, containerization strategies, and complex dependency resolution workflows that demonstrate the intricate nature of modern robotics development environments. The implementation began with establishing remote SSH connectivity from a Windows development machine located at IP address 192.168.137.1 to the target Raspberry Pi system at 192.168.137.19, utilizing the username raj for remote access authentication. The initial system environment revealed a Debian GNU/Linux system running kernel version 6.12.47+rpt-rpi-v8 with aarch64 architecture, indicating proper ARM64 support necessary for modern ROS2 implementations and ensuring compatibility with contemporary containerization technologies that require specific processor architectures for optimal performance characteristics.

The foundational system preparation phase involved comprehensive package management through apt package manager operations, beginning with sudo apt update commands to refresh package repositories and sudo apt full-upgrade operations to ensure all system components were current and properly maintained. The upgrade process identified twenty-four packages requiring updates, including critical components such as Chromium browser packages version 1:140.0.7339.207-1~deb13u1+rpt1, OpenSSL cryptographic libraries version 3.5.1-1+deb13u1 providing essential security functionality, and various Raspberry Pi-specific utilities including raspi-utils version 20251002-1, pipanel version 1:2.3 for system panel management, and wf-panel-pi version 1.5 window manager components that provide

desktop environment functionality specific to Raspberry Pi hardware configurations. The total download requirement reached 166 MB with processing involving chromium-l10n localization packages supporting multiple language configurations, chromium-common shared components providing core browser functionality, and chromium-sandbox security modules ensuring proper browser isolation and security boundary enforcement. Essential development tools were installed through a comprehensive command sequence including git version control system version 1:2.47.3-0+deb13u1 for source code management, build-essential compilation toolchain providing GCC compiler infrastructure, cmake build system generator enabling cross-platform compilation workflows, python3-pip package installer version 25.1.1+dfsg-1 for Python package management, python3-venv virtual environment creator version 3.13.5-1 for isolated Python development environments, i2c-tools version 4.4-2 for Inter-Integrated Circuit communication debugging and configuration, python3-smbus version 4.4-2 providing Python I2C library bindings for hardware interface programming, ca-certificates version 20250419 for secure communications certificate management, and curl version 8.14.1-2 command-line HTTP client for network operations and file transfers.

Hardware configuration procedures included enabling I2C bus functionality through `raspi-config nonint do_i2c 0` command, which programmatically enabled I2C hardware interface without requiring interactive configuration menu navigation, ensuring automated deployment capabilities for production environments. User permission management involved adding the `raj` user account to the `i2c` group through `usermod -aG i2c $USER` command, ensuring proper access privileges for I2C device communication and maintaining security boundaries while providing necessary hardware access rights. The `i2c-dev` kernel module was loaded using `sudo modprobe i2c-dev` command to establish low-level device driver support, enabling direct hardware communication pathways between user-space applications and I2C peripheral devices. I2C bus scanning was performed using `i2cdetect -y 1` command, revealing device detection at address `0x29` on the primary I2C bus, indicating successful hardware communication pathway establishment and confirming proper electrical connectivity and device enumeration on the I2C interface, which suggests the presence of connected sensor hardware or other I2C-compatible peripheral devices that could be utilized for additional sensor integration or system monitoring functionality.

Docker containerization infrastructure was implemented to provide isolated execution environments for ROS2 operations, ensuring reproducible deployment scenarios and dependency isolation between different software components. Initial `docker.io` installation encountered package resolution challenges requiring manual repository updates and dependency resolution procedures that demonstrate the complexity of modern container ecosystem management. The installation process involved downloading and configuring twenty-two packages totaling 57.6 MB of compressed data, including `runc` container runtime version 1.1.15+ds1-2+b4 providing low-level container execution capabilities, `containerd` daemon version 1.7.24~ds1-6+b4 for container lifecycle management, `iptables` firewall management version 1.8.11-2 for network security enforcement, `docker-buildx` build enhancement tools version 0.13.1+ds1-3 enabling advanced container build capabilities, and various Perl library dependencies including `libintl-perl` version 1.35-1, `libproc-processtable-perl` version 0.636-1+b3, and `libterm-readkey-perl` version 2.38-2+b4 for advanced container operations and system process management functionality. Docker service activation was accomplished through `systemctl status docker` commands, confirming active service status with process ID 2557 and successful daemon initialization, establishing proper `systemd` integration for automatic service startup and management capabilities. User privilege configuration required adding the `raj` user to the `docker` group and system reboot to apply group membership changes effectively, ensuring proper container access permissions while maintaining system security boundaries and preventing unauthorized container operations.

Docker image procurement involved multiple attempts to obtain compatible ARM64 ROS2 Humble images, demonstrating the architectural complexity of cross-platform container deployment in heterogeneous computing environments. Initial attempts with `osrf/ros:humble-desktop` image resulted in architecture mismatch errors, specifically `exec /ros_entrypoint.sh: exec format error` when attempting to execute AMD64 images on ARM64 hardware, highlighting the importance of architecture-specific container images in modern deployment workflows. This necessitated sourcing `arm64v8/ros:humble` image variant specifically compiled for ARM64 architecture, requiring 10 layers of container filesystem components including base Ubuntu filesystem structures, ROS2 core components providing middleware functionality, and various middleware packages totaling several gigabytes of container data that must be transferred and cached locally for efficient container instantiation. Container instantiation utilized complex parameter sets including interactive terminal allocation through `-it` flags enabling user interaction capabilities, privileged access through `--privileged` flag granting extended system access rights, host networking through `--network host` parameter enabling direct network interface access, volume mounting for persistent storage through `-v ~/ros2_ws:/root/ros2_ws` parameter ensuring data persistence across container lifecycle operations, and USB device

passthrough through `--device=/dev/ttyUSB0` parameter for direct hardware access to YDLiDAR sensors, demonstrating comprehensive hardware integration capabilities within containerized environments.

YDLiDAR hardware detection was verified through comprehensive kernel message analysis using `dmesg` commands, revealing USB device enumeration with specific hardware identifiers including `idVendor=10c4` identifying Silicon Labs manufacturer, `idProduct=ea60` specifying CP2102 device model, and `bcdDevice=1.00` indicating firmware version information, corresponding to Silicon Labs CP2102 USB to UART Bridge Controller with serial number 0001 providing unique device identification for multi-device environments. Device driver loading was confirmed through `usbserial` and `cp210x` module registration messages indicating proper kernel driver binding, establishing `/dev/ttyUSB0` device node creation with proper permissions configured as `crw-rw----` with root ownership and dialout group access rights, and symbolic link establishment in `/dev/serial/by-id/` directory structure providing persistent device naming through hardware-specific identifiers that remain consistent across system reboots and hardware reconnection events. The device enumeration process revealed proper USB subsystem integration with hardware monitoring capabilities including undervoltage detection messages indicating power supply monitoring functionality, demonstrating comprehensive system health monitoring and hardware status reporting capabilities essential for reliable robotics system operation in production environments.

ROS2 workspace initialization involved creating hierarchical directory structures using `mkdir -p ~/ros2_ws/src` commands to establish colcon build system compatible workspace layout, following ROS2 community conventions for project organization and build system integration. Initial colcon build operations resulted in empty package builds with zero packages processed, confirming proper build system installation and configuration while establishing baseline functionality for subsequent package compilation and dependency management operations. Source code acquisition involved cloning YDLiDAR ROS2 driver repositories from GitHub, specifically attempting to clone humble branch from `YDLIDAR/ydlidar_ros2.git` repository, which failed due to missing humble branch availability, demonstrating the importance of branch compatibility verification in distributed version control workflows. Alternative repository `YDLIDAR/ydlidar_ros2_driver.git` was successfully cloned, containing 128 objects with 816.07 KiB data transfer completing at 773.00 KiB/s transfer rate, including 93 objects enumerated during remote repository analysis, 63 compressed objects during transfer optimization, and successful delta resolution for 74 objects ensuring complete source code integrity and proper version control metadata preservation for future development and maintenance operations.

Compilation dependency resolution proved challenging when initial colcon build operations failed due to missing `ydlidar_sdk` package configuration files, indicating incomplete dependency chain resolution in the ROS2 ecosystem. The error messages specifically indicated `CMAKE_MODULE_PATH` searches for `Findydlidar_sdk.cmake` and `ydlidar_sdk-config.cmake` files, requiring YDLiDAR SDK installation as prerequisite dependency for successful ROS2 package compilation. This necessitated cloning YDLiDAR-SDK repository containing 1728 objects with 11.82 MiB total size transferred at 2.23 MiB/s, demonstrating comprehensive SDK implementation including sample code providing usage examples, documentation for developer reference, and platform-specific build configurations supporting multiple operating system environments and hardware architectures. The SDK repository structure included extensive C++ source code implementations, header files providing API definitions, CMake build configuration files enabling cross-platform compilation, and example applications demonstrating proper SDK usage patterns for various LiDAR sensor configurations and operational modes.

Build system configuration encountered complications when `cmake` commands were not found within the container environment, requiring manual CMake installation procedures that highlight the complexity of dependency management in containerized development environments. The resolution involved downloading CMake version 3.27.5 source code from GitHub releases, requiring 10.47 MB download with comprehensive compilation process involving bootstrap configuration, dependency analysis, and multi-stage build procedures. CMake bootstrap process involved extensive C++ compilation with numerous source files including `cmAddCompileDefinitionsCommand.cxx` for build definition management, `cmAddCustomCommandCommand.cxx` for custom build step integration, `cmAddCustomTargetCommand.cxx` for target configuration, and hundreds of additional components providing comprehensive build system functionality including cross-platform abstraction, dependency resolution, generator support for multiple build systems, and integration capabilities with various development tools and IDEs. The compilation process utilized GNU toolchain with `g++` compiler version providing C++17 support and `gmake` build processor enabling parallel compilation workflows, involving thousands of object file compilations with extensive header file dependencies including platform-specific abstractions for Linux, Windows, and macOS operating systems, utility library integrations providing file system operations, network communication capabilities, and cryptographic functionality essential for secure build operations and package management.

Python environment management complications arose due to externally-managed-environment restrictions imposed by modern Python installations following PEP 668 specifications, designed to prevent conflicts between system-managed packages and user-installed packages. The system prevented pip installations using standard pip install commands, requiring virtual environment creation through `python3 -m venv` commands or `--break-system-packages` override flags that bypass safety restrictions at the risk of system stability. This represented security-focused Python ecosystem changes designed to prevent system-wide package conflicts and maintain distribution package integrity while providing clear separation between system-managed Python environments and user-specific development environments. Alternative installation approaches involved creating isolated Python virtual environments using `python3 -m venv ~/venv` command sequences creating independent Python interpreter instances with dedicated package installation directories, or utilizing distribution-provided packages through apt package manager ensuring compatibility with system-wide dependency management and security update mechanisms.

Advanced CMake compilation involved extensive library building including `cmzlib` compression library providing data compression capabilities for build artifact optimization, `cmnhttp2` HTTP/2 protocol implementation enabling modern network communication protocols, `cmexpat` XML parsing library supporting configuration file processing, and `cmzstd` compression algorithms providing high-performance data compression for build caching and artifact management. The build process included numerous utility components such as `cmllibuv` asynchronous I/O library enabling non-blocking network and file system operations, `cmlihash` cryptographic hash functions providing data integrity verification and security validation, and comprehensive platform-specific abstractions for Windows, macOS, and Linux operating systems ensuring cross-platform compatibility and consistent build behavior across different development environments. Compilation progress indicated percentage completion tracking through automated build system monitoring, requiring substantial processing time for complete CMake installation on ARM64 hardware due to architectural performance characteristics and extensive source code base requiring compilation of thousands of source files with complex dependency relationships and optimization requirements.

SSL/TLS cryptographic support was ensured through `libssl-dev` package installation version `3.5.1-1+deb13u1`, providing development headers and libraries necessary for secure network communications including HTTPS protocol support, certificate validation mechanisms, and cryptographic algorithm implementations. The installation process involved 3.384 MB download with 17.0 MB installation footprint, including comprehensive cryptographic algorithm implementations supporting RSA, ECDSA, and EdDSA key formats, certificate validation mechanisms for PKI infrastructure support, and security protocols essential for secure ROS2 communications and package downloads from remote repositories requiring authenticated and encrypted data transfer operations. The SSL library installation also provided support for modern cryptographic protocols including TLS 1.3, forward secrecy mechanisms, and certificate transparency extensions ensuring robust security capabilities for production robotics deployments requiring secure communication channels and data protection compliance.

Container exit and host system operations demonstrated workflow transitions between containerized ROS2 environments and native Raspberry Pi system operations, showing the hybrid approach combining containerized isolation with host system integration capabilities. The process involved container disconnection with `client_loop: send disconnect: Connection reset` messages indicating network connection termination, followed by host system package management and build tool installations enabling development workflows that leverage both containerized consistency and native system performance characteristics. This hybrid approach allowed leveraging containerized ROS2 environments while maintaining host system tool accessibility for development operations, providing flexibility for complex development workflows requiring both isolation and integration capabilities depending on specific task requirements and performance considerations.

Native system CMake installation procedures involved bootstrap configuration establishing build environment parameters, parallel compilation using `make -j4` for quadcore CPU utilization maximizing build performance through parallel processing capabilities, and system-wide installation through `sudo make install` commands ensuring proper library path configuration and executable accessibility. CMake version verification confirmed successful installation with `cmake --version` commands returning proper version identification CMake 3.27.5, indicating successful compilation and installation of build system infrastructure. The installation process required substantial compilation time due to ARM64 architecture performance characteristics and extensive source code base requiring processing of multiple utility libraries including compression algorithms, network protocols, and cryptographic functions essential for modern build system operations and secure development workflows.

Python virtual environment creation addressed externally-managed-environment restrictions through `python3 -m venv ~/venv` command sequences, creating isolated Python installations with independent package management capabilities

that prevent conflicts with system-managed Python packages. Virtual environment activation involved source `~/venv/bin/activate` commands establishing separate Python interpreter instances with dedicated package installation directories, PATH modification for tool accessibility, and environment variable configuration ensuring proper isolation from system Python installation. This approach resolved pip installation restrictions while maintaining system security and package isolation principles, enabling development workflows that require specific Python package versions without compromising system-wide Python environment stability and ensuring reproducible development environments across different system configurations and deployment scenarios.

Colcon build system installation required alternative approaches due to package availability limitations in standard repositories, necessitating installation through Python package management systems or manual source compilation procedures. Installation attempts involved both pip-based installations within virtual environments ensuring package isolation and manual source code compilation procedures providing maximum compatibility with system-specific configurations and requirements. The `colcon-common-extensions` package provided essential build system functionality for ROS2 workspace management, including parallel compilation support enabling efficient multi-package build operations, dependency resolution ensuring proper build order and requirement satisfaction, and automated testing framework integration supporting continuous integration workflows and quality assurance processes essential for production robotics software development and deployment validation.

YDLiDAR SDK compilation involved traditional CMake build procedures with `mkdir build && cd build` directory creation establishing out-of-source build configuration preventing source tree contamination, `cmake ..` configuration generation analyzing dependencies and creating build system files, `make` compilation execution processing source code and generating binary artifacts, and `sudo make install` system installation copying libraries and headers to system directories enabling system-wide accessibility. The SDK installation provided essential runtime libraries including sensor communication protocols, data processing algorithms for point cloud generation and filtering, and sensor calibration utilities supporting various YDLiDAR sensor models and configurations. The SDK also included device driver interfaces enabling low-level hardware communication, configuration management utilities for sensor parameter adjustment, and diagnostic tools for troubleshooting hardware connectivity and performance issues.

Final ROS2 workspace compilation involved comprehensive dependency resolution with `ydliidar_sdk` availability enabling successful `ydliidar_ros2_driver` package compilation, demonstrating proper integration between SDK libraries and ROS2 middleware infrastructure. The colcon build system processed package dependencies ensuring proper compilation order, generated build configurations optimized for target architecture and system capabilities, and produced installable ROS2 packages with proper integration into ROS2 distribution frameworks including message definitions, service specifications, and node executable generation. Source installation through `source install/setup.bash` commands established runtime environment variables including `ROS_DOMAIN_ID`, `ROS_NAMESPACE`, and library path configurations necessary for ROS2 node execution, discovery mechanisms, and inter-node communication protocols enabling distributed robotics system operation across multiple computing nodes and sensor platforms.

Multi-LiDAR configuration considerations involved addressing hardware addressing through USB device enumeration ensuring unique device identification, USB device enumeration order management preventing device assignment conflicts during system startup, and simultaneous device access requirements enabling concurrent sensor operation for comprehensive environmental perception. The implementation supported dual YDLiDAR units through separate USB interface connections with distinct device node creation enabling individual sensor control and configuration, and concurrent access capabilities allowing parallel data acquisition and processing workflows essential for real-time robotics applications requiring high-frequency sensor updates and low-latency perception processing. Device management involved `udev` rule creation for consistent device naming ensuring predictable device paths across system reboots, access permission configuration for multi-user environments enabling collaborative development workflows, and device monitoring capabilities for hardware failure detection and automatic recovery procedures.

Advanced container configuration involved persistent volume mounting for workspace preservation across container restarts ensuring development continuity and data persistence, USB device passthrough for direct hardware access enabling sensor communication within containerized environments, privileged container execution for low-level hardware control providing necessary system privileges for device driver interaction, and host networking for seamless ROS2 communications between containers and host systems enabling distributed computing architectures and multi-node robotics system deployment. These configurations enabled comprehensive development environments with full hardware access capabilities while maintaining isolation benefits of containerization including dependency isolation, reproducible environments, and deployment consistency across different hardware platforms and system configurations.

The implementation process concluded with successful ROS2 workspace setup containing compiled YDLiDAR driver packages providing sensor integration capabilities, configured development environments with CMake build tools enabling cross-platform development workflows, established container infrastructure with ARM64 compatibility supporting modern deployment patterns, and verified hardware connectivity with dual YDLiDAR sensor support enabling advanced perception capabilities for autonomous navigation and mapping applications. The comprehensive installation provided foundation for advanced robotics applications including simultaneous localization and mapping operations utilizing sensor fusion algorithms, autonomous navigation implementations supporting waypoint navigation and obstacle avoidance, and sensor fusion algorithms utilizing dual YDLiDAR sensor configurations for enhanced environmental perception and robotic system reliability. The documented process represents complete end-to-end implementation of production-ready ROS2 development environment specifically optimized for YDLiDAR-based robotics applications on ARM64 hardware platforms, demonstrating successful integration of modern containerization technologies with traditional robotics development workflows while addressing architecture-specific compatibility requirements, dependency resolution challenges inherent in cross-platform robotics development environments, and hardware integration complexities typical of modern sensor-rich robotics systems requiring real-time performance and reliable operation in diverse environmental conditions.

Conclusion:

The rover is fully operational as a manually controlled platform, with all sensors calibrated and performing reliably. While the hardware has met and even exceeded design expectations, the software-driven goal of full autonomy remains a work in progress. The current system provides a robust foundation for future enhancements, ensuring that with further development in navigation algorithms and sensor integration, also all the components received were handled properly and are functional; the rover is well-positioned to achieve fully autonomous exploration in complex outdoor environments.



Team fortrans Information			
Parag Krishna Nikum	RCPIT, Shirpur	7387650405	nikumparag@gmail.com
Raj Ashok Sonawane	RCPIT, Shirpur	9307133716	mrrajsonawane@gmail.com